

Universal Hashing

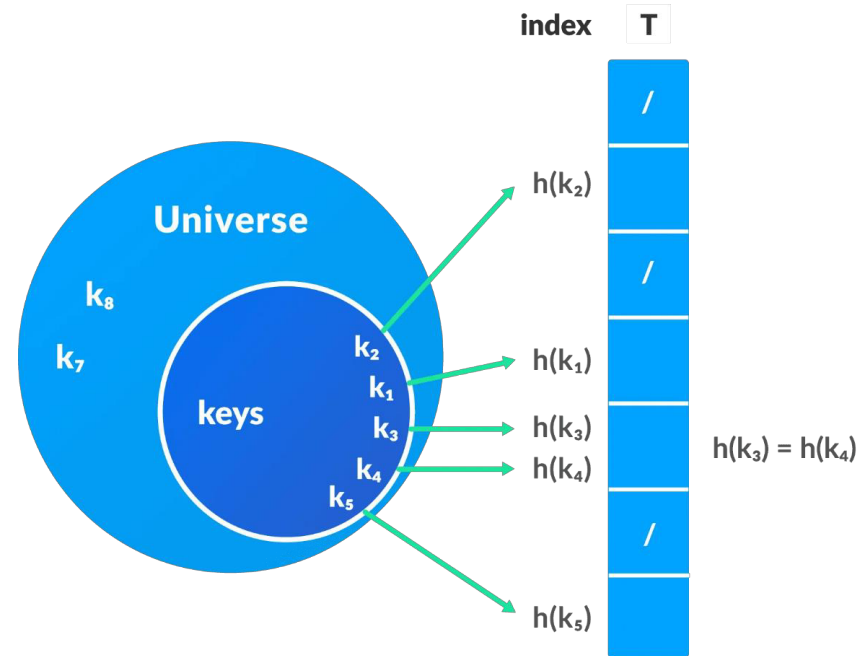
COMPSCI 753

Zijian Huang

The University of Auckland

Outline

- Universal Hashing Functions
- Desirable Performance Guarantee



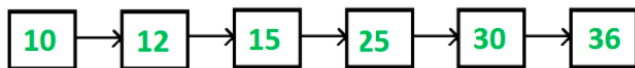
Dictionary Data Type

- **Dictionary problem:** Given a universe U of possible elements, maintain a subset $S \subseteq U$ supporting fast searching, inserting, deleting in S (in **constant** time)
- Main dictionary operations
 - **Insert(x)**: insert element $x \in U$ to S
 - **Delete(x)**: remove element x from S if $x \in S$
 - **Search(x)**: return true if $x \in S$ and false otherwise
- **Applications**
 - Search data in File Systems, Databases, Google Search, etc.

Dictionary Problem

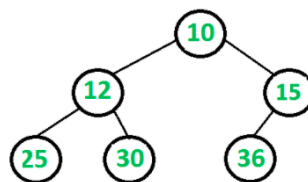
- **Challenges:**
 - The size of universe U is too large to store in an array of size $|U|$
 - Insert/delete/search operations on large sets take time
- Can we use the following data structures?

Linked List



Space	$O(\text{Length of the List})$
Search time	$O(\text{Length of the List})$

Binary Search Tree



$O(\# \text{ nodes in BST})$
$O(\text{Height of BST})$

Bitvector

00000000001010010000...

$O(U)$
$O(1)$

Hashing: A Randomized Implementation of Dictionaries

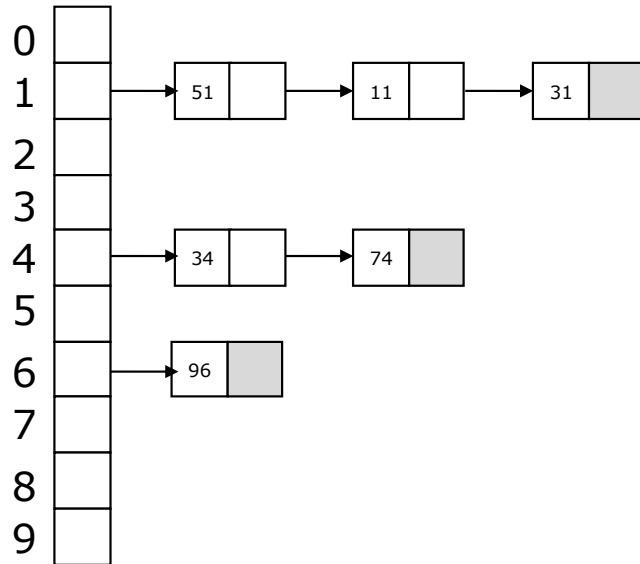
- Hash function $h: U \rightarrow \{0, 1, \dots, n - 1\}$
- Hashing
 - Create an array A of **length n**
 - For any operation on element $x \in S$:
 - Find the element's position using hash function $h(x)$
 - Operate on the array $A[h(x)]$
- Collision
 - Two different elements x and y have the same hash value: $h(x) = h(y)$
 - **Chaining**: $A[i]$ is a linked list containing all elements x where $h(x) = i$

E.g., Chain Hashing

$U = \{0, 1, \dots, 99\}$

$h(x) = x \% \text{table_size}$

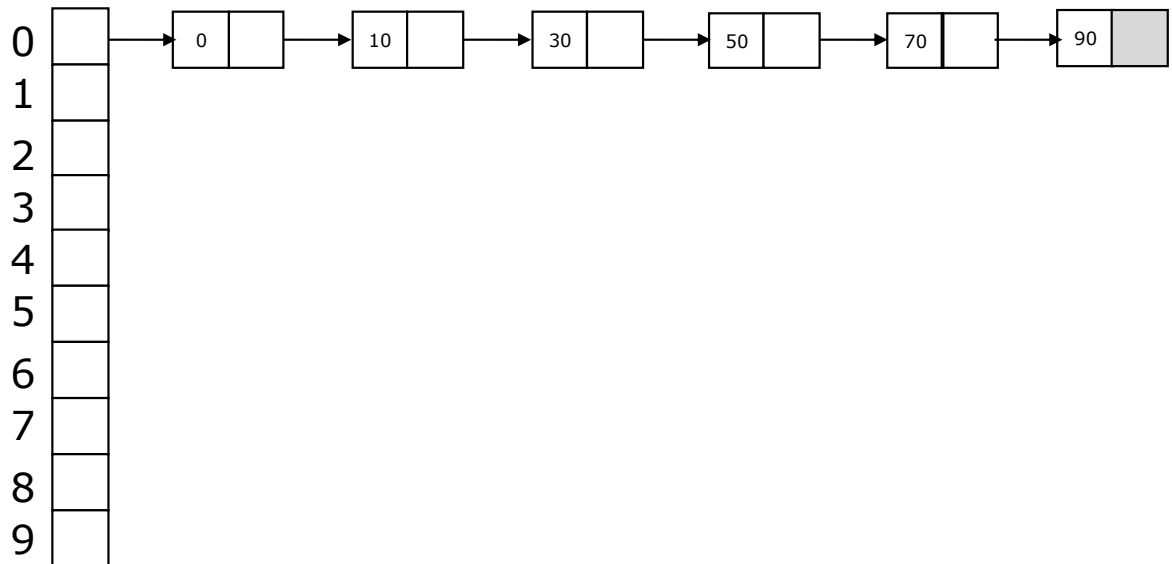
$S = \{96, 31, 11, 74, 51, 34\}$



$U = \{0, 1, \dots, 99\}$

$h(x) = x \% \text{table_size}$

$S = \{90, 70, 50, 30, 10, 0\}$ ← **worst case set of data**



Dictionary Operations

- **Search(x):**
 - **Compute** $h(x)$, then **scan** through the list $A[h(x)]$. Return true if finding x , and false otherwise.
- **Insert(x):**
 - **Compute** $h(x)$, then **scan** through the list $A[h(x)]$ to find x . If x is not in the list, add x to the front of the list. Otherwise, do nothing.
- **Delete(x):**
 - **Compute** $h(x)$, then **scan** through the list $A[h(x)]$ to find x . If x is in the list, remove x from the list. Otherwise, do nothing.
- Time complexity: $O(1 + \text{length of } A[h(x)])$
 - ↑ Compute $h(x)$
 - ↑ scan

Deterministic Hash Function

- For **any choice of h** , we can find a set whose elements have the same hash values (map to one slot)
- We end up with a single **linked list of size n** containing all elements. Hence, it takes **$O(n)$** time for the operations
- **Solutions:**
 - Assume the input S is random
 - **Ideal hash function:** Map m elements **uniformly at random** to n slots

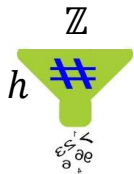
Hashing Performance

- Ideal hash function:
 - Any m elements are **uniformly distributed** to n slots
 - Average length of chain = m/n
 - Choose $n \sim m$, we expect $O(1)$ time per insert, delete, search operation
- We cannot have an ideal hash function for all possible sets of elements (true randomness might not exist)
- Approach: Choose the hash function **h at random** from a hash family **H**

Universal Hashing (Carter–Wegman'77)

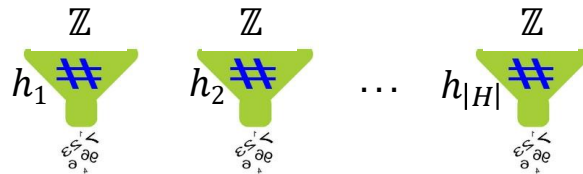
- A universal family of hash functions H is a set of hash functions h mapping the universe U to a set $\{0, 1, \dots, n-1\}$ s.t.
 - For any different pair $x \neq y$: $\Pr_h[h(x) = h(y)] \leq 1/n$, i.e., collision probability is very tiny
 - Can efficiently choose a hash function h at **random**
 - Can compute position $h(x)$ efficiently in constant time

worst case set of data



Case: 1 hash function h
 $|U|$ items all map to the same hash value

worst case set of data



Case: family of hash functions H
 $|U|$ items all map to the same hash value $\rightarrow 1/|H|$

Example: Universal Hashing

- Java String hashCode computes the address of a string using the Unicode values for each character ($A - Z$ corresponds to $65 - 90$, $a - z$ to $97 - 122$).
- For $s = s[0] \cdots s[n - 1]$ use integer arithmetic to calculate the hash via

$$s[0] \cdot 31^{n-1} + s[1] \cdot 31^{n-2} + \cdots + s[n - 1]$$

- Can you find two 2-character strings with the same hashCode? Why might cause a problem in practice?

$$h(bC) = 98 \times 31 + 67 = 97 \times 31 + 31 + 67 = 97 \times 31 + 98 = h(ab)$$

Example: Universal Hashing

- Universal property: If the collision probability $\Pr_h[h(x) = h(y)] \leq 1/n$ for any pair $x \neq y$

$U = \{a, b, c, d, e, f\}, \rightarrow \{0,1\}, n = 2$

	a	b	c	d	e	f
$h_1(x)$	0	1	0	1	0	1
$h_2(x)$	0	0	0	1	1	1

	a	b	c	d	e	f
$h_1(x)$	0	1	0	1	0	1
$h_2(x)$	0	0	0	1	1	1
$h_3(x)$	0	0	1	0	1	1
$h_4(x)$	1	0	0	1	1	0

$H = \{h_1, h_2\}$

$$\Pr_{h \in H}[h(a) = h(b)] = 1/2$$

$$\Pr_{h \in H}[h(a) = h(c)] = 1$$

$$\Pr_{h \in H}[h(a) = h(d)] = 0$$

...

$H = \{h_1, h_2, h_3, h_4\}$

$$\Pr_{h \in H}[h(a) = h(b)] = 1/2$$

$$\Pr_{h \in H}[h(a) = h(c)] = 1/2$$

$$\Pr_{h \in H}[h(a) = h(d)] = 1/2$$

$$\Pr_{h \in H}[h(a) = h(e)] = 1/2$$

$$\Pr_{h \in H}[h(a) = h(f)] = 0$$

...

← Not Universal



← Universal



单哈希

Example: Universal Hash Family for Integer

- We construct a universal hash function h that hashes an integer $x \in \mathbb{Z}$ to a set of $\{0, 1, \dots, n-1\}$ as follows (e.g., $n = 5,000$)
 - Pick a large prime number $p \geq n$ (e.g., $p = 999,331$)
 - Pick a random pair of integers $0 < a < p$, $0 \leq b < p$ (e.g., $a = 199, b = 1200$)

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod n$$

- Universal hash family: $H = \{h_{a,b} : 0 < a < p, 0 \leq b < p\}$
- Proof of the universal property
 - **[Book]** The 13.6 section of Algorithm Design
 - **[Paper]** Carter & Wegman: “Universal Classes of Hash Functions”. STOC’77

Example: 990 keys; $p = 991$; $m = 701$

$$h_{5,800}(x) = ((5x + 800) \bmod 991) \bmod 701$$

Key	10	700	55	330	900
Index	149	336	84	468	?

e.g.,

$$h_{5,800}(330) = \underbrace{((5 \times 330 + 800) \bmod 991)}_{1650} \bmod 701 = \underbrace{((2450 \bmod 991))}_{468} \bmod 701 = 468$$

Example: Universal Hash for a Set of Integers

- We construct a universal hash function h that hashes **a set of integers** $X = \{x_1, \dots, x_d\}$ to a set $\{0, 1, \dots, n-1\}$ as follows
 - Choose independently at **random** d hash functions $h_1, \dots, h_d$ from a universal hash family H

$$h(X) = h_1(x_1) + \dots + h_d(x_d) \bmod n$$

- Proof of universal property (Carter–Wegman'77)
- Practical hash function:** For a large prime p and $0 < a_i < p$ ($\forall 0 < i \leq d$), and $0 \leq a_0 < p$
 - $h_{a_0, \dots, a_d}(X) = ((a_0 + a_1x_1 + a_2x_2 + \dots + a_dx_d) \bmod p) \bmod n$

Performance Analysis: Dictionary Operations

- Time complexity: $O(1 + \text{length of } A[h(x)])$
- **Assumption:** $|S| \leq n$ at all time, hence our dictionary uses $O(n)$ space
- The expectation of the length of the linked list $A[h(x)]$
 - Let random variable $X_y = 1$ if an item y in $S \setminus \{x\}$ that collide with x (i.e., $h(y) = h(x)$), and 0 otherwise
 - **Question:** what is the expected number of items in S that collide with x
 - Poof.

$$\begin{aligned} \mathbf{E} [\text{length of } A[h(x)]] &= 1 + \mathbf{E}[\sum_{y \in S \setminus \{x\}} X_y] = 1 + \sum_{y \in S \setminus \{x\}} \mathbf{E}[X_y] &< \text{linearity of expectation} \\ &= 1 + \sum_{y \in S \setminus \{x\}} \Pr[X_y = 1] &< \mathbf{E}[X_y] = 1 \times \Pr[X_y = 1] + 0 \times \Pr[X_y = 0] \\ &\leq 1 + (|S| - 1) * \frac{1}{n} < 2 &< \frac{1}{n} \text{ due to universal hashing} \end{aligned}$$

- Time complexity of dictionary operations, in expectation, is $O(1)$

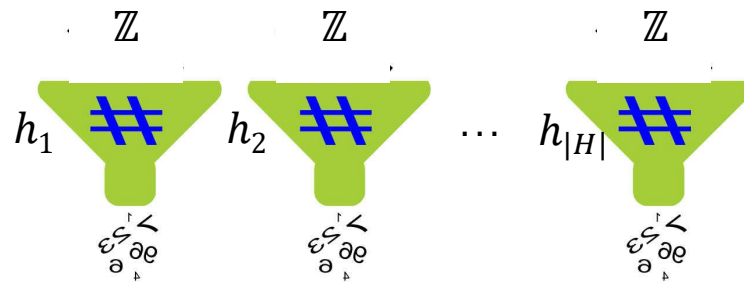
Table Representations: Comparative Performance

* : Retrieve, Update

Operation	Representation of a table of size m		
	Sorted array	AVL tree	Hash table
Initialise	$O(m)$	$O(1)$	$O(m)$
Is full?	$O(1)$	$O(1)$	$O(1)$
Search*	$O(\log m)$	$O(\log m)$	$O(1)$
Insert	$O(m)$	$O(\log m)$	$O(1)$
Delete	$O(m)$	$O(\log m)$	$O(1)$

Summary

- **Universal hashing:** Given a random choice of a hash function h from the universal family such that $h: U \rightarrow \{0, 1, \dots, n-1\}$
 - $h(x)$ spreads the data items out over the range of possible hash values
 - $h(x)$ takes less storage than the original data U
- Desirable Performance Guarantee
 - **$O(n)$ space** and **$O(1)$ expected time** per dictionary operation (**add, remove, search**)
- Expectation is due to the randomness of selection of hash functions



Resources

- **[Book]** Chapter13 “Algorithm Design” by Jon Kleinberg and Éva Tardos. [[LINK](#)]